

Pipes

A pipe transforms a value **right inside the template**, using the `|` symbol — without touching the actual value stored in your component.

1 · THE PROBLEM

Raw data doesn't always look presentable

We have

```
price = 49.9;
<p>{{ price }}</p>
// renders as: 49.9
```

We want

```
price = 49.9;
<p>{{ price | currency }}</p>
// renders as: $49.90
```

We need to format data for humans — dates, prices, text case — without changing the underlying value stored in the component.

2 · BUILT-IN PIPES

Angular gives you the common ones for free

PIPE	EXAMPLE	RESULT
currency	{{ 49.9 currency }}	\$49.90
date	{{ today date:'longDate' }}	August 24, 2026
uppercase	{{ name uppercase }}	AHMED
lowercase	{{ email lowercase }}	ahmed@site.com
percent, decimal, json	same syntax	niche / debugging

Without a pipe, you'd write formatting logic (`Intl.NumberFormat`, date parsing...) by hand inside every component that needs it.

3 · CUSTOM PIPES

When built-in pipes aren't enough

```
@Pipe({ name: 'truncate' })
export class TruncatePipe implements PipeTransform {
  transform(value: string, limit = 20) {
    return value.length > limit
      ? value.slice(0, limit) + '...'
      : value;
  }
}

{{ description | truncate:15 }}
// "Angular is a po..."
```

A custom pipe is a class with a `transform()` method — reusable anywhere in the app. Real-world: product descriptions, comment previews, table cells with limited width.

One product card, three pipes, zero extra component code

```
<h3>{{ productName | uppercase }}</h3>
<p class="price">{{ price | currency }}</p>
<p class="date">Added {{ addedDate | date:'mediumDate' }}</p>
```

```
// Result: USB-C HUB · $34.50 · Added Aug 24, 2026
```

Notice the component never formats anything itself — `productName`, `price`, and `addedDate` stay plain, simple values. All the formatting lives in the template.